

HDOC – Experiment 14

Question

Write a menu-driven C program using switch-case to input a positive integer and perform one of the following operations based on the user's choice:

- 1. Check whether all its digits are unique.**
- 2. Find the frequency of each digit from 0 to 9.**

❖ Understanding the problem

Input

- Positive Integer
- User's Menu Choice

Output

Option 1

- Unique, if every digit appears only once.
- Not Unique, if any digit appears more than once.

Option 2

- Frequency of each digit from 0 to 9.

❖ Test case table

Test case	Input (choice, number)	Expected output	Actual output	Result
1	1, 12345	Unique	Same	Pass
2	1,12234	Not unique	Same	Pass
3	2, 112333	Frequency: 1→2, 2→1, 3→3	Same	Pass
4	2, 908070	0→3, 7→1, 8→1, 9→1	Same	Pass
5	1, 9876543210	Unique	Same	Pass

❖ Algorithm

1. Start.
2. Display the menu with two options.
3. Read the user's choice.
4. Read the positive integer.
5. Initialize an array freq[10] with all elements equal to 0.
6. Copy the number into a temporary variable.
7. Extract each digit using % 10.
8. Increase the corresponding frequency in the array.
9. Remove the last digit by dividing the number by 10.
10. Repeat until all digits are processed.
11. According to the selected menu:-
 - Case 1: Check whether any frequency is greater than 1.
 - If yes, display Not Unique.
 - Otherwise, display Unique.
 - Case 2: Display the frequency of digits 0 to 9.
12. If the menu choice is invalid, display an error message.
13. Stop.

❖ Program code in C

```
#include <stdio.h>

int main()
{
    int choice;
    long long number, temp;
    int freq[10] = {0};
    int digit;
    int i;
    int unique = 1;

    printf("==== MENU =====\n");
    printf("1. Check Whether Digits are Unique\n");
    printf("2. Find Frequency of Digits (0-9)\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    printf("Enter a positive integer: ");
    scanf("%lld", &number);

    temp = number;

    if(temp == 0)
        freq[0] = 1;
```

```
while(temp > 0)
{
    digit = temp % 10;
    freq[digit]++;
    temp = temp / 10;
}
```

```
switch(choice)
```

```
{
```

```
    case 1:
```

```
        for(i = 0; i < 10; i++)
```

```
        {
```

```
            if(freq[i] > 1)
```

```
            {
```

```
                unique = 0;
```

```
                break;
```

```
            }
```

```
        }
```

```
        if(unique)
```

```
            printf("\nOutput: Unique\n");
```

```
        else
```

```
            printf("\nOutput: Not Unique\n");
```

```
        break;
```

```
    case 2:
```

```

printf("\nFrequency of Digits:\n");

for(i = 0; i < 10; i++)
{
    printf("freq_%d = %d\n", i, freq[i]);
}

break;

default:

    printf("\nInvalid Choice!\n");
}

return 0;
}

```

❖ Compile, Execute and Evaluate

Test case 1

```

(kali㉿kali)-[~/Desktop]
$ ./unique
===== MENU =====
1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)
Enter your choice: 1
Enter a positive integer: 12345

Output: Unique

```

Test case 2

```
(kali㉿kali)-[~/Desktop]
$ ./unique
===== MENU =====
1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)
Enter your choice: 1
Enter a positive integer: 12234

Output: Not Unique
```

Test case 3

```
(kali㉿kali)-[~/Desktop]
$ ./unique
===== MENU =====
1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)
Enter your choice: 2
Enter a positive integer: 112333

Frequency of Digits:
freq_0 = 0
freq_1 = 2
freq_2 = 1
freq_3 = 3
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0
```

Test case 4

```
(kali㉿kali)-[~/Desktop]
$ ./unique
===== MENU =====
1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)
Enter your choice: 2
Enter a positive integer: 908070

Frequency of Digits:
freq_0 = 3
freq_1 = 0
freq_2 = 0
freq_3 = 0
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 1
freq_8 = 1
freq_9 = 1
```

Test case 5

```
(kali㉿kali)-[~/Desktop]
$ ./unique
===== MENU =====
1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)
Enter your choice: 1
Enter a positive integer: 9876543210

Output: Unique
```

❖ Compilation and execution screenshot

```
(kali㉿kali)-[~/Desktop]
$ vi unique.c

(kali㉿kali)-[~/Desktop]
$ gcc unique.c -o unique

(kali㉿kali)-[~/Desktop]
$ ./unique
===== MENU =====
1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)
Enter your choice: 1
Enter a positive integer: 12345

Output: Unique

(kali㉿kali)-[~/Desktop]
$ ./unique
===== MENU =====
1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)
Enter your choice: 1
Enter a positive integer: 12234

Output: Not Unique

(kali㉿kali)-[~/Desktop]
$ ./unique
===== MENU =====
1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)
Enter your choice: 2
Enter a positive integer: 112333

Frequency of Digits:
freq_0 = 0
freq_1 = 2
freq_2 = 1
freq_3 = 3
```

```
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0
```

```
(kali@kali)-[~/Desktop]
$ ./unique
```

```
===== MENU =====
```

1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)

Enter your choice: 2

Enter a positive integer: 908070

Frequency of Digits:

```
freq_0 = 3
```

```
freq_1 = 0
```

```
freq_2 = 0
```

```
freq_3 = 0
```

```
freq_4 = 0
```

```
freq_5 = 0
```

```
freq_6 = 0
```

```
freq_7 = 1
```

```
freq_8 = 1
```

```
freq_9 = 1
```

```
(kali@kali)-[~/Desktop]
$ ./unique
```

```
===== MENU =====
```

1. Check Whether Digits are Unique
2. Find Frequency of Digits (0-9)

Enter your choice: 1

Enter a positive integer: 9876543210

Output: Unique